

1) a)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct elemento{
    int valor;
    struct elemento *proximo;
} elemen;

typedef struct{
    elemen *inicio;
}lista;

//funcao para inicializar a lista
lista *initlist(){
    lista *l=(lista *)malloc(sizeof(lista));
    //verificar se ocorreu erro
    if(l==NULL){
        printf("ERRO NA INICIALIZACAO DA LISTA");
        exit(1);
    }

    //o inicio aponta para o null
    l->inicio=NULL;
    //retorna a lista
    return l;
}

//funcao para inserir na lista
void inserirList(lista *l, int valor){
    //verifica se a lista existe
    if(l==NULL){
        printf("LISTA NAO INICIALIZA");
        exit(1);
    }

    //alocar um novo elemento
    elemen *novo=(elemen *)malloc(sizeof(elemen));
    if(novo==NULL){
        printf("ERRO NA CRIACAO DE UM NOVO ELEMENTO");
        exit(1);
    }
}
```

```

        //inserir
        novo->valor=valor;
        novo->proximo=NULL;

        novo->proximo=l->inicio;
        l->inicio=novo;
    }

    //funcao para inserir na lista
    void inserirListNoMeio(lista *l, int posicao, int valor){
        //verifica se a lista existe
        if(l==NULL){
            printf("LISTA NAO INICIALIZA");
            exit(1);
        }

        //alocar um novo elemento
        elemen *novo=(elemen *)malloc(sizeof(elemen));
        if(novo==NULL){
            printf("ERRO NA CRIACAO DE UM NOVO ELEMENTO");
            exit(1);
        }

        //inserir
        novo->valor=valor;
        novo->proximo=NULL;

        elemen *atual=l->inicio;

        //nao preocupo pela posicao pois ja verifiquei no main
        for(int i=0; i<posicao-1; i++){
            //percorrer
            atual=atual->proximo;
        }

        novo->proximo=atual->proximo;
        atual->proximo=novo;

    }

    //funcao para inserir no fim da lista
    void inserirFimList(lista *l, int valor){
        //verificar se a lista existe

```

```

    if(l==NULL){
        printf("A LISTA NAO INICIALIZADA");
        exit(1);
    }

    //alocar um novo elemento
    elemen *novo=(elemen *)malloc(sizeof(elemen));
    if(novo==NULL){
        printf("ERRO NA CRIACAO DE UM NOVO ELEMENTO");
        exit(1);
    }

    novo->valor=valor;
    novo->proximo=NULL;

    if(l->inicio==NULL){
        //lista vazia, apenas inserir
        novo->proximo=l->inicio;
        l->inicio=novo;
    }else{
        //usar atual para percorrer
        elemen *atual=l->inicio;
        //percorrer ate quando o prox é null
        while(atual->proximo!=NULL){
            atual=atual->proximo;
        }

        //achou o atual que seria o ultimo elemento
        novo->proximo=atual->proximo;
        atual->proximo=novo;
    }
}

//inserir por ordem
void inserirOrdemList(lista *l, int valor){
    //verificar se existe a lista
    if(l==NULL){
        printf("LISTA NAO INICIALIZADA!");
        exit(1);
    }

    //alocar um novo elemento
    elemen *novo=(elemen *)malloc(sizeof(elemen));

```

```

        if(novo==NULL){
            printf("ERRO NA CRIACAO DE UM NOVO ELEMENTO");
            exit(1);
        }

        novo->valor=valor;
        novo->proximo=NULL;

        if(l->inicio==NULL || l->inicio->valor>valor){
            //nao ha nada na lista
            novo->proximo=l->inicio;
            l->inicio=novo;
        }else{
            //elemen atual para percorrer
            elemen *atual=l->inicio;

while(atual->proximo!=NULL&&atual->proximo->valor<valor){
            atual=atual->proximo;
        }

        novo->proximo=atual->proximo;
        atual->proximo=novo;

    }
}

//funcao para excluir elemento
void removeElemen(lista *l, int valor){
    //verificar se existe a lista
    if(l==NULL){
        printf("LISTA NAO INICIALIZADA");
        exit(1);
    }

    //criar os elementos para percorrer(anterior e atual)
    elemen *atual=l->inicio;
    elemen *anterior=NULL;

    //percorrer ate o final da lista, e ate chegar valor
    while(atual!=NULL||atual->valor!=valor){
        anterior=atual;
        atual=atual->proximo;
    }
}

```

```

    }

    //caso nao encontrado
    if(atual==NULL){
        printf("VALOR %d NAO ENCONTRADO\n", valor );
    }

    //se o elemento é o primeiro elemento
    if(anterior==NULL){
        l->inicio=atual->proximo;
    }else{
        anterior->proximo=atual->proximo;
    }
    free(atual);
}

//FUNCAO PARA CONTAR ELEMENTOS
int contagemList(lista *l){
    //verificar se existe a lista
    if(l==NULL){
        printf("LISTA NAO INICIALIZADA");
        exit(1);
    }

    //elemento temporario para percorrer ate o NULL
    elemen *temp=l->inicio;

    int cont=0;
    while(temp!=NULL){
        cont++;
        temp=temp->proximo;
    }

    //retornar o valor
    return cont;
}

//EXIBIR A LISTA
void printList(lista *l){
    //verificar se existe a lista
    if(l==NULL){
        printf("LISTA NAO INICIALIZADA");
    }
}

```

```

        exit(1);
    }

    //elemento atual p percorrer
    elemen *atual=l->inicio;

    while(atual!=NULL){
        printf("\t%d\t", atual->valor);
        atual=atual->proximo;
    }

    printf("\n");
}

//BUSCAR NA LISTA
void searchList(lista *l, int valor){
    //verificar se ja existe a lista
    if(l==NULL){
        printf("LISTA NAO INICIALIZADA");
        exit(1);
    }

    //elemento para percorrer
    elemen *atual=l->inicio;

    //percorrer ate o null e ate chegar no valor
    while(atual != NULL && atual->valor != valor){
        atual=atual->proximo;
    }

    if(atual == NULL){
        printf("VALOR %d NAO ENCONTRADO");
    }

    if(atual->valor==valor){
        printf("VALOR %d ENCONTRADO");
    }
}

//BUSCA NA LISTA ORDENADA
void searchListOrdenada(lista *l, int valor) {
    // Verificar se a lista existe
    if (l == NULL) {

```

```

        printf("LISTA NAO INICIALIZADA\n");
        exit(1);
    }

    // Elemento para percorrer
    elemen *atual = l->inicio;

    // Para se chegar ao fim OU se encontrar um valor maior ou
    igual ao procurado
    while (atual != NULL && atual->valor < valor) {
        atual = atual->proximo;
    }

    // Se saiu do loop e o atual nao e NULL, checa se e
    exatamente o valor
    if (atual != NULL && atual->valor == valor) {
        printf("VALOR %d ENCONTRADO\n", valor);
    } else {
        printf("VALOR %d NAO ENCONTRADO\n", valor);
    }
}

//funcao para free tudo
void freeList(lista *l){
    //verificar se ja existe a lista
    if(l==NULL){
        printf("LISTA NULA");
        exit (1);
    }

    elemen *atual = l->inicio;    // O ponteiro "atual" começa no
    início da lista
    elemen *temp = NULL;          // Ponteiro temporário para
    guardar o elemento atual antes de avançar para o próximo
    while(atual != NULL){
        temp = atual;              // Guardar o ponteiro para o
        elemento atual
        atual = atual->proximo; // Avançar para o próximo
        elemento
        free(temp);              // Liberar a memória do elemento
        atual
    }
    free(l);
}

```

```

}

int main(){
    lista *l = initlist();

    //colocar na lista
    inserirOrdemList(l, 10);
    inserirOrdemList(l, 4);
    inserirOrdemList(l, 8);
    inserirOrdemList(l, 7);

    //contar
    int cont = contagemList(l);

    printf("\n-----\n");
    printf("a lista tem %d elementos\n", cont);

    //print
    printf("a lista é: \t");
    printList(l);
    printf("\n");

    int valor, posicao;
    printf("digite o valor que quer inserir:\n");
    scanf("%d", &valor);
    printf("\n digite depois de quantos elemento voce quer
colocar:\n ");
    scanf("%d", &posicao);

    if(posicao>cont){
        printf("posicao invalida!");
        return 1;
    }

    //inserir na lista
    inserirListNoMeio(l, posicao, valor);
    printf("\n-----\n");
    printf("a lista apos inserir o valor: \t");
    printList(l);
    return 0;
}

```


b)

```
void ordenarList(lista *l){
    //verificar se existe a lista
    if(l==NULL){
        printf("LISTA NAO INICIALIZADO");
        exit(1);
    }
    elemen *ptr1;
    elemen *ptr_fim=NULL;
    int trocou;

    do{
        trocou=0;
        ptr1=l->inicio;
        while(ptr1 != NULL && ptr1->proximo != ptr_fim){
            if(ptr1->valor > ptr1->proximo->valor){
                int temp = ptr1->valor;
                ptr1->valor=ptr1->proximo->valor;
                ptr1->proximo->valor=temp;
                trocou=1;
            }
            ptr1 = ptr1->proximo;
        }
        ptr_fim=ptr1;
    }while(trocou==1);
}
```

c)

```
void Inverte(lista *l) {
    // 1. Verifica se a lista foi inicializada
    if (l == NULL) {
        printf("LISTA NAO INICIALIZADA!\n");
        return; // Como a funcao e void, usamos apenas return
    }

    // 2. Se a lista estiver vazia ou tiver so 1 elemento, nao
    // precisa inverter
    if (l->inicio == NULL || l->inicio->proximo == NULL) {
        return;
    }

    // 3. Inicializa os ponteiros auxiliares para a inversao
}
```

```
elemen *anterior = NULL;
elemen *atual = l->inicio;
elemen *proximo = NULL;

// 4. Percorre a lista invertendo os apontadores (celulas de
posicao)
while (atual != NULL) {
    proximo = atual->proximo; // Salva o resto da lista
    atual->proximo = anterior; // Inverte o ponteiro do
elemento atual
    anterior = atual;          // Avança o anterior
    atual = proximo;          // Avança o atual
}

// 5. Ajusta o ponteiro de inicio da lista para o novo
primeiro elemento
l->inicio = anterior;
}
```